

NumPy Cheat Sheet

SESSIONS 1-8 · QUICK REVISION REFERENCE

Arrays · Creation · Attributes · Operations
Math & Stats · Indexing · Manipulation · Utilities

Session 1 · Intro to NumPy

PENDING

Why analysts use NumPy over plain Python lists.

WHAT IS NUMPY

Library for fast numerical computing in Python, built around the `ndarray` object.

ARRAYS VS LISTS

List	Array
Mixed types	Single dtype
Slow loops	Vectorized C ops
More memory	Compact, fixed-size

MEMORY EFFICIENCY

Fixed dtype + contiguous memory block → far less overhead per element than a Python list of objects.

VECTORIZATION

Apply an operation to a whole array at once (in C), avoiding explicit Python `for` loops → much faster.

Session 3 · Attributes & Reshaping

PENDING

ATTRIBUTES

- `.ndim` — number of dimensions
- `.shape` — tuple of dimension sizes
- `.size` — total number of elements
- `.dtype` — data type of elements
- `.itemsize` — bytes per element
- `.nbytes` — total bytes = size × itemsize

RESHAPING

- `.reshape(shape)` — new shape, same data (view)
- `.ravel()` — flatten to 1D (view, if possible)
- `.flatten()` — flatten to 1D (always a copy)
- `.resize(shape)` — reshape in-place, can change size

TRANSPOSE

- `.T` — transpose shorthand
- `np.transpose(arr)` — transpose (custom axis order)

Session 2 · Creating Arrays

PENDING

BASIC CREATION

- `np.array(list)` — from a Python list/tuple
- `np.arange(start, stop, step)` — range-like sequence
- `np.linspace(start, stop, num)` — evenly spaced, fixed count
- `np.logspace(start, stop, num)` — evenly spaced in log scale

FILLED ARRAYS

- `np.zeros(shape)` — all zeros
- `np.ones(shape)` — all ones
- `np.empty(shape)` — uninitialized (garbage values)
- `np.eye(n)` / `np.identity(n)` — identity matrix

RANDOM ARRAYS

- `np.random.rand(shape)` — uniform [0,1)
- `np.random.randn(shape)` — standard normal
- `np.random.randint(lo, hi, size)` — random ints
- `np.random.choice(arr, size)` — random sample from array
- `np.random.seed(n)` — reproducible randomness

Session 4 · Array Operations

PENDING

ARITHMETIC (ELEMENT-WISE)

- `+` add `-` sub `*` mul `/` div `//` floor-div `**` power `%` mod

COMPARISON (ELEMENT-WISE, RETURNS BOOL ARRAY)

- `==` `!=` `>` `<` `>=` `<=`

BROADCASTING

NumPy auto-expands smaller-shaped arrays across a larger array's dimensions so element-wise ops "just work" without manual loops.

Session 5 · Math & Stats

PENDING

MATH

- `np.add` `np.subtract` `np.multiply` `np.divide`
- `np.power` `np.mod` `np.sqrt` `np.exp` `np.log`
- `np.log10`

TRIGONOMETRIC

- `np.sin` `np.cos` `np.tan` `np.arcsin`
- `np.degrees` `np.radians`

ROUNDING

- `np.round` (nearest) · `np.floor` (down) · `np.ceil` (up)

STATISTICS

- `np.mean` `np.median` `np.std` `np.var` `np.min`
- `np.max` `np.sum` `np.cumsum` `np.prod`
- `np.percentile`

Session 6 · Indexing & Slicing

PENDING

- `arr[i]` / `arr[i,j]` — basic indexing 1D/2D/nD
- `arr[start:stop:step]` — slicing
- `arr[-1]` — negative indexing (from the end)
- `arr[::2]` — step slicing
- `arr[[0,2,4]]` — fancy indexing (list/array of indices)
- `arr[arr > 5]` — boolean indexing (condition mask)
- `np.where(cond, x, y)` — conditional selection
- Slices return **views** (share memory); fancy/boolean indexing returns **copies**.

Session 7 · Array Manipulation

PENDING

JOINING

- `np.concatenate([a,b], axis)` — join along existing axis
- `np.vstack([a,b])` — stack rows (new axis 0)
- `np.hstack([a,b])` — stack columns (axis 1)
- `np.dstack([a,b])` — stack along depth (axis 2)

SPLITTING

- `np.split(arr,n)` — equal-size split (errors if uneven)
- `np.array_split(arr,n)` — split, allows uneven parts

ADD / REMOVE

- `np.insert(arr,idx,val)`
- `np.append(arr,val)`
- `np.delete(arr,idx)`
- `.copy()` — independent array. `.view()` — shares memory with original.

Session 8 · Utility Functions

PENDING

SORTING & UNIQUENESS

- `np.unique(arr)` — sorted unique values
- `np.sort(arr)` — sorted copy
- `np.argsort(arr)` — indices that would sort the array

SEARCH & LOGIC

- `np.where(cond)` — indices where condition is true
- `np.nonzero(arr)` — indices of nonzero elements
- `np.any(arr)` / `np.all(arr)` — any/all True

VALIDITY CHECKS

- `np.isnan(arr)` · `np.isinf(arr)` · `np.isfinite(arr)`
- `np.clip(arr,min,max)` — cap values to a range

HANDLING NAN

- `np.nan` — the NaN constant
- `np.nanmean` `np.nanstd` `np.nansum` — ignore NaNs in the calculation

Session status: all 8 sessions currently PENDING · use this sheet to preview what's coming and revise fast once each session is delivered